PROJECT PROPOSAL

Pulse CBD

A live, multimodal transit heartbeat for Sydney's CBD, projected onto a physical 3D scale model

PREPARED	STATUS	DATA SOURCE
2026-08-22	Software complete & live-verified; physical installation pending	Transport for NSW Open Data Hub

---

> Pulse CBD turns Sydney's live public transport network into a physical, ambient object: every train, metro, light rail vehicle, ferry and bus currently moving through the CBD appears as a point of light, projected onto a tabletop scale model of the city, moving along the model's actual streets and rail corridors in real time. This proposal covers the chosen presentation modality and why it fits the project's goals, the data sources and analytical approach behind it, and a sketch of the technical architecture that delivers it.

1. Overview of the Chosen Modality

The core presentation choice is **projection onto a physical 3D scale model** of the Sydney CBD, rather than a conventional 2D map on a screen. A downward-facing projector, mounted above a laser-cut or 3D-printed model of the CBD's streets and rail corridors, displays the renderer's output directly onto the model's surface. A calibrated homography maps real-world coordinates onto projector pixel space, so a vehicle's glowing dot lands on the model's actual street or rail line, not just on an abstract map background.

Why this modality suits the project's goals

- **Spatial legibility that a flat map can't match.** A transit network is fundamentally a spatial, physical thing — tracks, streets, harbours, interchanges. A tangible model gives viewers an intuitive frame of reference (this street, that harbour crossing) that a 2D map on a screen, however well-designed, always renders one level more abstractly.
- **Ambient and glanceable, not interaction-dependent.** The installation is legible from across a room, at a glance, without anyone needing to zoom, pan, or tap anything. It works as passive ambient information — closer to a living display than a dashboard someone has to actively query.
- **The city reads as alive.** Watching light move along real streets in real time communicates the rhythm and scale of a transit network — rush-hour density, harbour ferries, the CBD rail core —

far more viscerally than a table of numbers or even an animated map. That emotional legibility is a project goal in itself, not just a side effect.

- **Clean separation between "what to show" and "where to show it."** Projecting onto a passive physical surface keeps the model itself simple — no embedded electronics, no per-LED wiring, no fixed resolution baked into the hardware. All of the intelligence stays in software. This also means the exact same renderer runs identically on a plain screen during development, and the physical installation is a calibration step, not a rebuild — the project doesn't have to choose between "build for a screen" and "build for a model," because the architecture treats projection as one interchangeable output of the same pipeline (see Section 3).
- **Cheap to iterate, easy to recalibrate.** Because alignment is a software homography rather than a physical/optical property of the display, the model can be moved, the projector repositioned, or the bounding box adjusted, and recalibration is a two-minute interactive process (drag crosshairs, enter known coordinates, save) rather than a hardware change.

Alternatives considered

Two other physical-projection modalities were evaluated against the fixed scale-model approach. Both use live computer vision to sense the physical surface itself, rather than treating it as a known, static shape:

Modality	What it is	Why it wasn't chosen
Live-sensed sand table	A depth camera (e.g. Kinect) continuously reads the surface of a sandpit shaped by hand, and topography/contour lines are projected back onto it in real time — the museum "AR sandbox" pattern.	Its whole value is letting the surface change — the terrain is the variable being explored. The CBD's streets and rail corridors are fixed, known geometry; there is nothing to reshape, so a continuous depth-sensing loop would be solving a problem this project doesn't have. Sand also can't hold the fine, precise linear detail (a specific street, a specific rail line) that vehicles need to visibly track.
Sensed LEGO grid	A large modular LEGO baseplate, with block height and colour read back via stereo-photogrammetry or a homography-corrected camera, so the physical arrangement of blocks becomes live input to the system.	Reconfigurability is the feature, and again isn't needed here — the network doesn't move. The stud grid's fixed, blocky resolution is also a poor fit for Sydney's organic, non-orthogonal street layout and harbour edge, which a laser-cut or 3D-printed model can reproduce far more precisely at any resolution the fabrication method supports.

The common thread: both alternatives are built to sense a physical state that is *itself the input*, which is the right tool when the model's arrangement is the data. Here it isn't — the data is entirely external (live transit feeds) and the model exists purely as a stable, precise output surface. A fixed model needs calibrating once, via a homography anchored to a handful of registration marks (Section 3), and stays valid indefinitely; a camera-sensed surface needs an ongoing computer-vision pipeline — depth or stereo processing, drift-prone camera calibration, sensitivity to the very projector light it's trying to read through — for a capability the project doesn't use. That complexity

budget belongs on the data side, where the real hard problems already are (Section 2), not duplicated on the display side.

2. Data Sources and Analytics Approach

Data sources

Source	What it provides	Cadence
Static GTFS TfNSW Open Data Hub, Complete GTFS bundle	Routes, stops, trips, stop-time schedules, and route shapes (the actual geographic path of every service) for every operator in Greater Sydney	Refreshed daily
GTFS-Realtime — Vehicle Positions	Live lat/lon, bearing, and trip/route identifiers for every active vehicle, across 7 modes: Sydney Trains, Metro, NSW Trains, Buses, Ferries, and both Light Rail networks	Polled every 15s (the feed's own publish rate)
GTFS-Realtime — Trip Updates	Per-trip arrival/departure delay at the next scheduled stop, and trip cancellation status	Polled every 15s
GTFS-Realtime — Service Alerts	Active disruptions, with severity and the routes/stops they affect	Polled every 15s
TfNSW custom protobuf extension field 1007	Additional vehicle/carriage-level detail (occupancy, accessibility) beyond the standard GTFS-Realtime schema	Embedded in the above

Analytics approach

The static bundle covers the entire Greater Sydney network and the realtime feeds cover the entire state; the analytics pipeline's first job is narrowing that down to what the CBD model actually needs, then turning raw, imperfect field data into something a physical light can represent honestly.

- **Spatial scoping.** Static entities are filtered to a bounding box around the CBD before anything else happens, keeping the working dataset to a manageable size without discarding the schedule context (a trip's full stop sequence is retained even for the stops it makes outside the box, so a vehicle has history rather than appearing from nowhere).
- **Entity resolution across two independent systems.** The realtime feeds and the static schedule don't always agree on identifiers — a bus trip can be altered mid-route, and some rail/light-rail feeds use an entirely different internal identifier scheme than the published schedule. Resolution falls back through three strategies in order: exact trip match, route-level match, and finally a geometric match (see below) — each one labelled honestly rather than silently guessed.
- **Map matching.** Raw GPS positions are snapped onto the vehicle's actual route geometry, both to correct for GPS drift (so a train doesn't appear to be driving through a building) and, as a

secondary effect, to identify a vehicle's route purely by *where it is* when its reported identifiers don't resolve any other way.

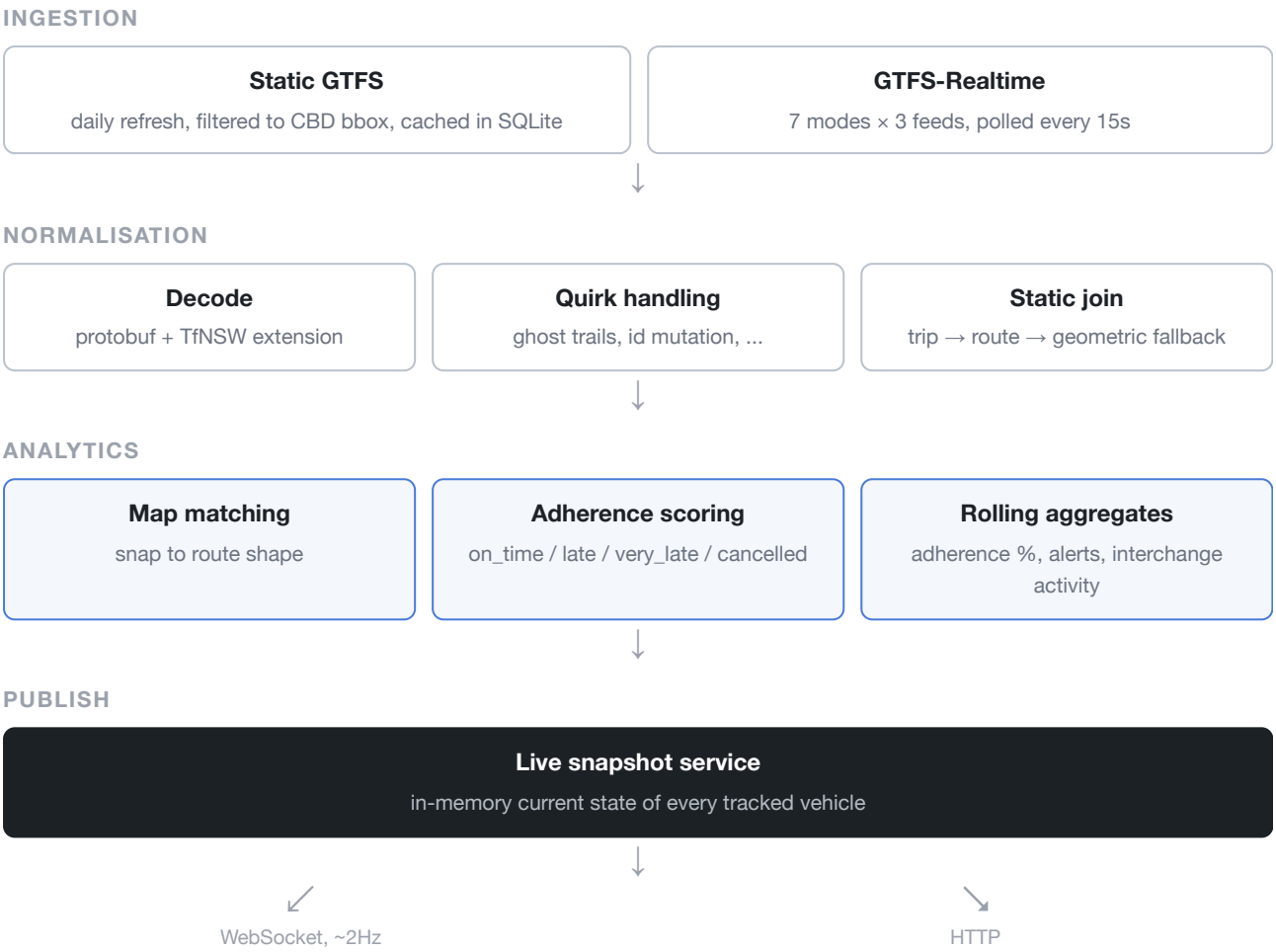
- **Schedule-adherence scoring.** Delay is computed from the Trip Updates feed's next-scheduled-stop event, not by comparing interpolated position against a timetable — the two data sources are kept separate on purpose. Each vehicle is classified into an adherence state using thresholds tuned per mode (metro and light rail run a tighter clock than intercity rail).
- **Temporal aggregation.** Individual vehicle states are rolled up into sliding-window statistics — on-time percentage by mode over the last 5, 15 and 60 minutes, and distinct-vehicle activity counts per interchange over the last 5 minutes — to surface network-level patterns that no single vehicle's state can show on its own.
- **Alert correlation.** Statewide service alerts are cross-referenced against the CBD's own route and stop sets, so only locally relevant disruptions surface.

DESIGN PRINCIPLE

The analytics layer computes and publishes discrete, ground-truth state on every poll; all smoothing, interpolation, and visual motion happen downstream in the renderer. Keeping the analytical core purely about "what is true right now" — rather than mixing in presentation concerns — keeps it deterministic and testable independent of how (or whether) it's ever displayed.

3. Technical Architecture Sketch

The system is built as two deliberately separate layers: a headless **data engine** that has no knowledge of how — or whether — its output is ever displayed, and a **projection renderer** that consumes a clean, published stream. The data engine runs and is fully verifiable with no renderer attached at all.



Major components

Component	Responsibility
Static ingestion & cache	Fetches and spatially filters the GTFS bundle; persists to a queryable local store so the rest of the system never touches the network for schedule data
Realtime source abstraction	Polls the live feeds behind a swappable interface, so the identical pipeline can run against live data, a recorded replay, or synthetic test data
Decode & quirk handling	Turns raw protobuf into structured records and absorbs the real-world inconsistencies documented in Section 2
Normalisation & enrichment	Joins realtime to static, map-matches position, and scores schedule adherence per vehicle
Aggregate layer	Maintains the rolling network-level statistics behind the wall-panel dashboard

Live snapshot service	The single in-memory source of truth every downstream consumer reads from
Publish layer	Broadcasts a compact vehicle-state frame over WebSocket roughly twice a second, plus plain HTTP endpoints for the dashboard and debugging
Calibration & homography	An interactive tool for mapping real-world coordinates onto projector pixel space, persisted so the model stays aligned across restarts
Renderer	A lightweight browser client that draws vehicles as glowing, colour- and size-coded points with short motion trails, using whichever coordinate projection is currently active — a flat-screen fit during development, a calibrated homography once projected onto the physical model